

Futcamedic - Documento de Requisitos del Producto (PRD)

Sistema Completo de Gestión Para Academias de Fútbol

Equipo de Producto
Futcamedic

5 de mayo de 2026

PRD: Futcamedic - Sistema Multi-Tenant de Gestión de Academias de Fútbol

Declaración del Problema

Los propietarios de academias de fútbol que gestionan 50-500 alumnos luchan con herramientas fragmentadas: cuadernos de papel para asistencias, grupos de WhatsApp para programación, seguimiento manual de cobros en efectivo y hojas de cálculo Excel para registros de alumnos. Esta fragmentación causa pagos perdidos (15-25% de ingresos mensuales perdidos), seguimiento inexacto de asistencias (sin datos históricos), conflictos de programación entre categorías y comunicación retrasada con padres. Los propietarios de academias necesitan un sistema unificado donde puedan gestionar alumnos, rastrear asistencias, cobrar pagos y programar sesiones de entrenamiento desde un dispositivo móvil, con notificaciones automáticas a padres.

Criterios de Éxito

1. El propietario de la academia puede registrar la escuela e invitar a 3+ profesores en menos de 10 minutos después del primer inicio de sesión
2. Los profesores pueden tomar asistencias para 20+ alumnos en menos de 3 minutos por sesión de entrenamiento
3. La tasa de cobro de pagos aumenta un 30% dentro de 90 días debido a las notificaciones automáticas a padres
4. El 95% de las operaciones de la academia (asistencias, pagos, programación) se manejan dentro de la aplicación dentro de los 60 días de adopción
5. Cero filtración de datos entre escuelas (aislamiento multi-tenant verificado mediante políticas RLS)

Historias de Usuario

- Como **propietario de academia (admin)**, necesito registrar mi escuela e invitar profesores para que mi personal pueda comenzar a usar el sistema inmediatamente
- Como **profesor**, necesito tomar asistencias para mis categorías asignadas para que la academia rastree la participación de los alumnos con precisión
- Como **profesor**, necesito ver mi cronograma de entrenamientos para saber dónde y cuándo presentarme a las sesiones
- Como **padre**, necesito ver el historial de asistencias y pagos de mis hijos para mantenerme informado sobre su estado
- Como **propietario de academia**, necesito registrar pagos y notificar automáticamente a los padres para reducir la fricción en la cobranza
- Como **propietario de academia**, necesito crear eventos recurrentes para no programar manualmente cada sesión

Qué Está En Alcance

1. Registro de escuela multi-tenant con creación de usuario admin
2. Control de acceso basado en roles (super_admin, admin, profesor, padre, alumno)
3. Gestión de alumnos con información médica, vinculación de padres y seguimiento de estados (activo, beca, pendiente, inactivo)
4. Seguimiento de asistencias con funcionalidad de “pase de lista” en bloque
5. Registro de pagos (mensualidades de alumnos, nómina de profesores) con notificaciones push a padres
6. Gestión de eventos con motor de recurrencia (semanal, quincenal, mensual)
7. Gestión de categorías vinculadas a años de nacimiento con asignación de profesores
8. Gestión de sedes (campos de entrenamiento) para lugares de entrenamiento
9. Sistema de permisos granulares para profesores
10. Aplicación móvil (iOS/Android) con navegación por pestañas basada en rol
11. Aplicación web alternativa con funcionalidad idéntica
12. Sistema de notificaciones push vía Expo
13. Carga de fotos de perfil a Supabase Storage
14. Exportación de reportes financieros a PDF/Excel

Qué Está Fuera de Alcance

1. Procesamiento de pagos en línea (integración con Stripe, PayPal) — pagos registrados manualmente
2. Validación de ubicación GPS para toma de asistencias
3. Importación masiva CSV para alumnos
4. Comunicación con padres más allá de notificaciones de pago
5. Portal para alumnos (rol alumno existe pero UI no está implementada)
6. Soporte multi-idioma (actualmente solo español)
7. Modo sin conexión (requiere conexión a internet)
8. Integración con sistemas de calendario externos (Google Calendar, Outlook)
9. Recordatorios automáticos de pago (solo notificación al registrar manualmente)
10. Alertas de emergencia médica basadas en información médica del alumno

Criterios de Aceptación

Autenticación y Registro: - [] El registro de escuela crea usuario auth, registro de escuela y perfil de admin en <30 segundos - [] Email de confirmación enviado vía Supabase Auth al registrarse - [] Usuarios invitados reciben contraseña temporal y deben cambiarla en primer inicio de sesión - [] Token JWT validado en cada solicitud de API vía middleware `requireAuth` - [] Aislamiento multi-tenant impuesto vía middleware `requireTenant` en todas las rutas

Gestión de Alumnos: - [] Admin puede crear alumno con nombre, fecha nacimiento, información médica, vinculación de padre - [] Estado del alumno puede cambiarse: activo, beca, pendiente, inactivo - [] Foto del alumno cargada al bucket `avatars` de Supabase Storage - [] Padre solo puede ver sus propios hijos (filtrado vía `parent_id`) - [] Alumnos eliminados auditados en tabla `deleted_students` con seguimiento `deleted_by`

Asistencias: - [] Profesor ve solo entrenamientos para sus categorías asignadas - [] Guardado de asistencia en bloque con alternancia presente/ausente para cada alumno - [] Lógica de `upsert` previene registros duplicados de asistencia (único: `student_id`, `date`, `type`) - [] Entrenamiento marcado como `is_completed = true` después de guardar asistencia - [] Asistencia visible por alumno, padre, profesor y admin

Pagos: - ☐ Registro de pago para alumno individual o en bloque (multi-alumno) - ☐ División automática de monto cuando se seleccionan múltiples alumnos (pago grupal) - ☐ Notificación push enviada a padre(s) al registrar pago - ☐ Tipos de pago: mensualidad, pago_profesor - ☐ Reporte de pagos pendientes muestra alumnos sin pago en el mes actual - ☐ Exportar a PDF/Excel con filtros por mes y tipo

Eventos y Programación: - ☐ Creación de evento con recurrencia: semanal (seleccionar días), quincenal, mensual - ☐ Motor de recurrencia genera registros individuales de trainings - ☐ Eventos vinculados a categoría, sede y con hora de inicio - ☐ Tipos de evento: entrenamiento, partido - ☐ Cancelar sesión individual o evento completo con notificación a padres - ☐ Profesor ve solo entrenamientos para sus categorías

Permisos de Profesores: - ☐ Permisos granulares: can_manage_students, can_manage_events, can_view_finances, can_manage_payments, can_take_attendance, can_manage_categories - ☐ Permisos predeterminados configurados al invitar profesor - ☐ Admin puede actualizar permisos por profesor - ☐ UI del profesor se adapta basándose en banderas de permisos

Multi-Tenencia: - ☐ Todas las tablas tienen columna school_id - ☐ Políticas RLS imponen coincidencia de school_id en todas las consultas - ☐ Middleware de backend configura req.tenant con school_id, role, user_id - ☐ Cero filtración de datos entre tenants verificada vía pruebas manuales

Dependencias

1. **Proyecto Supabase:** Base de datos PostgreSQL con RLS habilitado, Auth configurado, bucket de Storage avatars creado
2. **Cuenta Render.com:** Para despliegue de API backend
3. **Cuenta Expo:** Para compilación de aplicación móvil y servicio de notificaciones push
4. **Node.js y npm:** Tiempo de ejecución de backend y gestión de paquetes
5. **Clave de Rol de Servicio Supabase:** Para operaciones de administración que omiten RLS en backend
6. **Variables de Entorno:** SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, EXPO_PUBLIC_SUPABASE_URL, EXPO_PUBLIC_SUPABASE_ANON_KEY, EXPO_PUBLIC_API_URL

Preguntas Abiertas

1. ¿Cuál es el número máximo de alumnos por academia antes de que el rendimiento se degrade?
2. ¿Debería el sistema soportar múltiples padres por alumno (madre + padre)?
3. ¿Qué sucede con los datos históricos cuando se elimina un profesor del sistema?
4. ¿Deberían detenerse los eventos recurrentes si se desactiva una categoría?
5. ¿Hay necesidad de planes de pago (cuotas) más allá de mensualidades?
6. ¿Cómo debe manejar el sistema las sustituciones de profesores para una sesión de entrenamiento?
7. ¿Deberían aplicarse límites de capacidad de sede al crear eventos?